Sustavi baza podataka

Predavanja

4. Obavljanje upita

(2. dio)

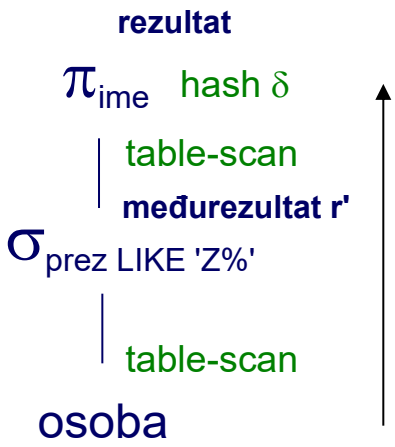
ožujak 2026.

Složeni izrazi relacijske algebre

- način prosljeđivanja međurezultata -

Način prosljeđivanja međurezultata

- rezultat složenog izraza relacijske algebre evaluira se kao niz pojedinačnih operacija relacijske algebre
- osim redoslijeda obavljanja operacija i odabira fizičkih operatora, planom se treba definirati i način postupanja s rezultatima pojedinačnih operatora (međurezultatima) jer rezultat jednog operatora predstavlja ulazni argument sljedećeg operatora



A) Materijalizacija (*materialization*)

- međurezultat se kao cjelina pohranjuje
  - ili u međuspremnike
  - ili u sekundarnu memoriju (ako je međurezultat prevelik za raspoložive međuspremnike). Problem: trošak U/I operacija radi zapisivanja (a kasnije opet čitanja) međurezultata

Primjer (sa slike):

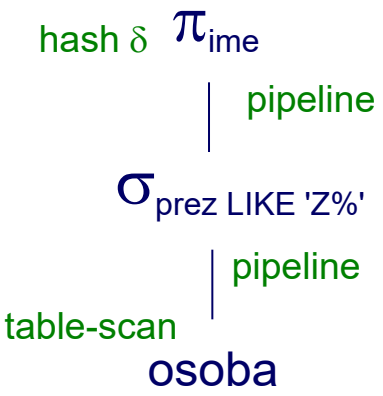
- obavi $\sigma_{prez\ LIKE\ 'Z\%'}$ i pohrani međurezultat r'
- obavi π_{ime} (r')

(materijalizirati ga u sekundarnoj memoriji)

Način prosljeđivanja međurezultata

B) Cjevovodi (*pipelining, stream-based processing*)

- fizički operator s niže razine (proizvođač, *producer*) producira pojedinačne n-torke koje koristi operator na višoj razini (potrošač, *consumer*)
  - *demand-driven (pulling)* - najčešća implementacija



Primjer:

- operator π zahtijeva od operatora σ sljedeću n-torku, operator σ zahtijeva od operatora *table-scan* sljedeću n-torku, operator *table-scan* čita n-torku i predaje u σ, σ obrađuje n-torku i predaje u π, π obrađuje n-torku i predaje kao rezultat

- operator selekcije, spajanje s ugniježđenim petljama, *bag* verzija unije, primjeri su operatora naročito pogodnih za primjenu u modelu cjevovoda
- neki operatori ne mogu izbjeći materijalizaciju: npr. vanjsko sortiranja s uparivanjem mora materijalizirati barem sortirane segmente. Ujedno, taj operator može prvu n-torku rezultata vratiti potrošaču tek kada započne posljednja faza uparivanja.

Prosljeđivanje rezultata i iteratori

- kod implementacije modela *pulling*, SUBP koristi poseban oblik sučelja koji se naziva *iterator*
- korištenjem uvijek istog sučelja, zakrivaju se interni implementacijski detalji fizičkog operatora. Time se omogućuje primjena, po potrebi, različitih operatora koji obavljaju istu operaciju, a implementirani su optimalno za svaki specifični slučaj
- Za svaki operator implementiraju se tri metode:
  - **`open()`**: inicijalizira stanje iteratora, alocira potrebnu memoriju (npr. ulazne i izlazne redove)
  - **`getNext()`**: operatoru-potrošaču vraća "sljedeću n-torku"
  - **`close()`**: oslobađa zauzete resurse

- iteratori se mogu izvršavati svaki unutar svojeg vlastitog procesa ili dretve
  - omogućuje se paralelno izvršavanje više fizičkih operatora istog upita
  - osim ako operator zahtijeva materijalizaciju međurezultata

Iterator *table scan*

Informativno

```
TableScan {
  open(r) {
    bR := the first block of r;
    tR := the first tuple of bR;
    FOUND := true;  /* global variable */
  }
  getNext() {
    if tR is past last tuple on block bR
      increment bR to the next block;
      if there is no next block in r
        FOUND := false;
        return;
      else
        tR := the first tuple of bR;
    return_tR := tR;
    increment tR to the next tuple of bR;
    return return_tR;
  }
  close() { ... }
}
```

- pseudokod je znatno pojednostavljen - nedostaju oznake programskih blokova {}, definicije varijabli, rukovanje praznim relacijama, alokacija memorije u open() i oslobađanje memorije u close(), ...
- vidi se da je i implementacija također vrlo pojednostavljena. Ovdje se npr. pretpostavilo da u zadanoj relaciji sigurno postoji barem jedna n-torka

Iterator *union (bag)*

Informativno

```
BagUnion {
  open(r, s) {
    tscanR := new TableScan(); tscanR.open(r);
    currentRel := r;
  }
  getNext() {
    if currentRel = r
      t := tscanR.getNext();
      if FOUND
        return t;
      else
        tscanR.close();
        currentRel := s;
        tscanS := new TableScan(); tscanS.open(s);
    return tscanS.getNext();
  }
  close() {
    tscanS.close(); ...
  }
}
```

$r \cup^B s$

Složeni izrazi relacijske algebre

- procjena troškova -

Složeni izrazi relacijske algebre - procjena troškova

Primjer:

```
SELECT *
   FROM stud AS r, ispit AS s
   WHERE mbr = mbrStud
     AND datIspit = '17.6.2008';
```

$\sigma_{datIsp='17.6.2008' \wedge mbr=mbrStud}(stud \times ispit)$

$\sigma_{datIsp='17.6.2008'}(stud \underset{mbr=mbrStud}{\triangleright\!\triangleleft} ispit)$

$stud \underset{mbr=mbrStud}{\triangleright\!\triangleleft} (\sigma_{datIsp='17.6.2008'}(ispit))$

ekvivalentni izrazi relacijske algebre

- procijeniti broj U/I operacija koje će se obaviti za svaki od navedenih ekvivalentnih izraza relacijske algebre

Složeni izrazi relacijske algebre - procjena troškova

Pretpostavke:

stud

- N(r) = 5 000
- B(r) = 500
- V(mbr, r) = 5000
- veličina n-torke t_r = 0.05 blokova

ispit

- N(s) = 100 000
- B(s) = 3000
- V(datIsp, s) = 200
- veličina n-torke t_s = 0.02 blokova

Uočiti: B(r) može biti veći od N(r) · veličina n-torke t_r jer blokovi ne moraju biti u cijelosti popunjeni n-torkama

- svi međurezultati se zapisuju u sekundarnu memoriju
- za spajanje (i Kartezijev produkt) koristiti *block nested-loop join*. Pretpostaviti najlošiji slučaj: na raspolaganju je samo M=3 (broj blokova glavne memorije).
- nad relacijama nema kreiranih indeksa
- primarni ključ relacije r je mbr

Složeni izrazi relacijske algebre - procjena troškova

$\sigma_{datIsp='17.6.2008' \wedge mbr=mbrStud}(stud \times ispit)$

- trošak za r_1 = stud × ispit - *block nested-loop*
  - čitanje: $B(r) + B(r) \cdot B(s) = 500 + 1.5 \cdot 10^6 \approx 1.5 \cdot 10^6$ U/I operacija
  - pisanje međurezultata (r_1):
    - $N(r_1) = N(r) \cdot N(s) = 500 \cdot 10^6$
    - veličina n-torke rezultata: 0.07 blokova (zašto?)
    - $B(r_1) = 35 \cdot 10^6$ blokova $\Rightarrow 35 \cdot 10^6$ U/I operacija
- trošak za $r_2 = \sigma_{datIsp='17.6.2008' \wedge mbr=mbrStud}(r_1)$ - *table-scan*
  - $35 \cdot 10^6$ U/I operacija
- ukupni trošak (bez troškova zapisivanja rezultata): $\approx 71.5 \cdot 10^6$ U/I operacija

Složeni izrazi relacijske algebre - procjena troškova

$$\sigma_{datIsp='17.6.2008'}(stud \underset{mbr=mbrStud}{\triangleright\!\triangleleft} ispit)$$

- trošak za $r_1 = stud \underset{mbr=mbrStud}{\triangleright\!\triangleleft} ispit$ - *block nested-loop join*
  - čitanje: $B(r) + B(r) \cdot B(s) = 500 + 1.5 \cdot 10^6 \approx 1.5 \cdot 10^6$ U/I operacija
  - pisanje međurezultata (r_1):
    - $N(r_1) =$ 100 000 - jer se svaka t_s može spojiti s najviše jednom t_r
    - veličina n-torke rezultata: 0.07 blokova
    - $B(r_1) = 7000$ blokova $\Rightarrow$ 7000 U/I operacija
- trošak za $r_2 = \sigma_{datIsp='17.6.2008'}(r_1)$ - *table-scan*
  - 7000 U/I operacija
- ukupni trošak (bez troškova zapisivanja rezultata): $\approx 1.51 \cdot 10^6$ U/I operacija

Složeni izrazi relacijske algebre - procjena troškova

$$stud \underset{mbr=mbrStud}{\triangleright\!\triangleleft} (\sigma_{datIsp='17.6.2008'}(ispit))$$

- trošak za $r_1 = \sigma_{datIsp='17.6.2008'}(ispit)$ - *table-scan*
  - čitanje: B(s) = 3000 U/I operacija
  - pisanje međurezultata (r_1):
    - $N(r_1)$ = 100 000 / V(datIsp, s) - pretp: jednolika distribucija po datIsp
    - veličina n-torke rezultata: 0.02 blokova
    - $B(r_1)$ = 10 blokova $\Rightarrow$ 10 U/I operacija
- trošak za r_2 = stud $\triangleright\!\triangleleft$ r_1 - *block nested-loop join*
  - $B(r_1) + B(r_1) \cdot B(r)$ = 5010 U/I operacija

- ukupni trošak (bez troškova zapisivanja rezultata): 8020 U/I operacija

- za vježbu: procijeniti trošak za svaki od prikazanih primjera ako su na raspolaganju M=102 blokova glavne memorije

Složeni izrazi relacijske algebre - procjena troškova

- trošak izvršavanja operacija relacijske algebre nad temeljnim relacijama i materijaliziranim međurezultatima ovisi o primijenjenom fizičkom operatoru
- u složenim izrazima relacijske algebre važna je i veličina međurezultata
- međurezultati se (u osnovi) pohranjuju kao neporedane datoteke
  - posljedica: broj U/I operacija koje se obavljaju nad međurezultatima je proporcionalan veličini međurezultata
- prethodni primjer pokazuje kako je za procjenu veličine međurezultata važan i broj n-torki i veličina n-torke
  - veličina n-torke je podatak lako dostupan iz rječnika podataka. Ako se radi o n-torci varijabilne duljine, može se koristiti podatak o prosječnoj veličini n-torke
  - broj n-torki u rezultatu obavljanja neke operacije nije moguće saznati prije nego se operacija obavi
- budući da se ne može unaprijed znati koliko će n-torki imati međurezultat, koriste se pravila za procjenu broja n-torki u rezultatu operacije relacijske algebre
- cilj procjene NIJE predvidjeti točan broj n-torki u međurezultatu

Procjena broja n-torki u rezultatu operacije rel. algebre

- za relaciju r(A, B, ...) u rječniku podataka pohranjeni su statistički podaci:
  - N(r) - broj n-torki relacije r
  - V(A, r) - broj različitih vrijednosti atributa A u relaciji r
    - $V(A, r) = \mathcal{G}_{COUNT(*)}(\pi_A(r))$
      - pri tome A može biti jedan atribut ili skup atributa
  - min(A, r), max(A, r) - najmanja i najveća vrijednost atributa A u relaciji r
  - histogrami

Jednostavna selekcija

Pravila za procjenu veličine rezultata nisu univerzalna. Ovdje je prikazan podskup pravila koja se koriste u jednom od sustava.

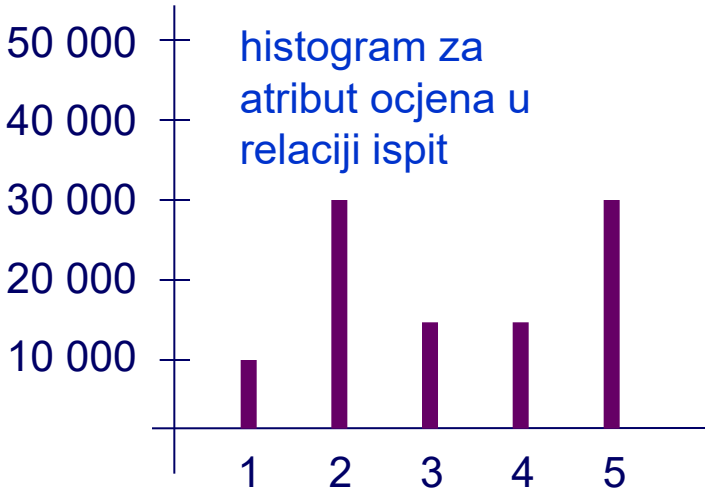
$\sigma_{A=v}(r)$

1. koristiti histogram (ako je na raspolaganju), inače
2. ako je V(A, r) poznat, uz pretpostavljenu jednoliku razdiobu vrijednosti atributa A

   procijenjeni broj n-torki = N(r) / V(A, r), inače
3. ako V(A, r) nije poznat: procijenjeni broj n-torki = N(r) / 10

Histogrami:

- za svaki raspon vrijednosti atributa A (granice raspona ovise o primijenjenoj rezoluciji) u rječniku podataka se evidentira broj n-torki, npr.



- ispit(mbrStud, datIspit, sifPred, ocjena)
  - N(ispit) = 100 000
  - V(ocjena, ispit) = 5

Procjena broja n-torki za $\sigma_{ocjena=2}$ (ispit)

- pomoću histograma: 30 000
- pomoću V(A, r): 20 000
- samo pomoću N(r): 10 000

Jednostavna selekcija

$\sigma_{A \leq v}(r)$ v je konstanta

- ako je vrijednosti v poznata u trenutku optimizacije
  - ako postoji histogram, koristiti histogram
  - inače, ako su vrijednosti min(A, r) i max(A, r) poznate

    ako je $v < \min(A, r)$ = 0

    ako je $v \geq \max(A, r)$ = N(r)

    ako je $\min(A, r) \leq v < \max(A, r)$

    = $N(r) \cdot (v - \min(A, r)) / (\max(A, r) - \min(A, r))$
- ako vrijednost v ili min(A, r), max(A, r) nisu poznate u trenutku optimizacije

  = N(r) / 3
  - u tom slučaju se isti izraz koristi i za nejednakosti oblika $<$, $\geq$, $>$

$\sigma_{A \textbf{ LIKE } izraz}(r)$ = N(r) / 5

Selekcija - uvjet selekcije s negacijom

$\sigma_{\neg F}(r)$

- ako se procijenjeni broj n-torki za operaciju $\sigma_F(r)$ označi s $N_F(r)$ tada je procijenjeni broj n-torki u rezultatu operacije $\sigma_{\neg F}(r)$
  $= N(r) - N_F(r)$

- prethodni izraz se može iskoristiti za procjenu npr.
  $\sigma_{A \neq v}(r) = \sigma_{\neg(A=v)}(r) \Rightarrow N(\sigma_{A \neq v}(r)) = N(r) - N(\sigma_{(A=v)}(r))$

- slično također za
  $\sigma_{A \text{ NOT LIKE izraz}}(r) = \sigma_{\neg(A \text{ LIKE izraz})}(r)$

- slično također za
  $\sigma_{A>v}(r) = \sigma_{\neg(A \leq v)}(r)$
  - osim ako v ili min(A, r), max(A, r) nisu poznati u trenutku optimizacije, tada se opet procjenjuje na N(r) / 3, a ne N(r) - N(r) / 3

Selektivnost predikata (*selectivity*)

- f(F, r): broj iz intervala [0, 1]
- što je selektivnost predikata F? Omjer broja n-torki iz r koje zadovoljavaju predikat F i ukupnog broja n-torki u r
- $f(F, r) = N(\sigma_F(r)) / N(r)$ $\Rightarrow N(\sigma_F(r)) = N(r) \cdot f(F, r)$

npr. za $\sigma_{A=v}(r)$

- ako je V(A, r) poznat, f(A=v, r) = N(r) / V(A, r) / N(r) = 1 / V(A, r)
- ako V(A, r) nije poznat, f(A=v, r) = N(r) / 10 / N(r) = 0.1

- u oba slučaja procijenjeni broj n-torki se izračunava na sljedeći način:
  - $N(\sigma_{A=v}(r)) = N(r) \cdot f(A=v, r)$

Složena selekcija - konjunktivna forma

$\sigma_{F_1 \wedge F_2 \wedge \ldots F_m}(r)$

- za svaki F_i se procjenjuje veličina rezultata $N_{F_i}(r) = N(\sigma_{F_i}(r))$
- $N_{F_i}(r)$ je procijenjeni broj n-torki za operaciju $\sigma_{F_i}(r)$
  - procjena se obavlja prema prethodno opisanim pravilima

- vjerojatnost da n-torka zadovoljava pojedinačni uvjet F_i je $N_{F_i}(r) / N(r)$
  - odnosno $f(F_i, r)$ - ovako je lakše napisati istu vrijednost
- ako su F_i nezavisni, vjerojatnost da n-torka zadovoljava sve F_i jest

  $$(N_{F_1}(r) / N(r)) \cdot (N_{F_2}(r) / N(r)) \cdot \ldots \cdot (N_{F_m}(r) / N(r))$$
  - odnosno $f(F_1, r) \cdot f(F_2, r) \cdot \ldots f(F_m, r)$
- procijenjeni broj n-torki u rezultatu je dakle

  $$N(r) \cdot (N_{F_1}(r) \cdot N_{F_2}(r) \cdot \ldots \cdot N_{F_m}(r)) / (N(r))^m$$
  - odnosno $N(r) \cdot f(F_1, r) \cdot f(F_2, r) \cdot \ldots f(F_m, r)$

Nezavisnost pojedinačnih uvjeta u složenim predikatima

- prethodno navedeni izrazi podrazumijevaju nezavisnost pojedinačnih uvjeta

osoba

mbr	pbrRod	pbrStan	spol
101	51000	51000	Ž
102	51000	51000	M
103	51000	51000	Ž
104	51000	21000	M
105	51000	51000	Ž
106	51000	51000	M
107	51000	51000	Ž
108	51000	51000	M
109	21000	21000	Ž
110	21000	21000	M
111	21000	21000	M
112	21000	21000	Ž
113	21000	21000	Ž
114	21000	21000	M
115	21000	51000	Ž
116	21000	21000	M

  - N(osoba) =16
  - V(pbrRod, osoba) = 2
  - V(pbrStan, osoba) = 2
- ocijeniti nezavisnost pojedinačnih uvjeta u složenim predikatima sljedećih algebarskih operacija

$$s = \sigma_{pbrRod=51000 \wedge pbrStan=21000}(osoba)$$

$$N(s) = 16 \cdot 0.5 \cdot 0.5 = 4$$

a koliko je zapravo n-torki u rezultatu ?

$$s = \sigma_{pbrRod=51000 \wedge spol='Ž'}(osoba)$$

$$N(s) = 16 \cdot 0.5 \cdot 0.5 = 4$$

a koliko je zapravo n-torki u rezultatu ?

Složena selekcija - disjunktivna forma

$\sigma_{F_1 \vee F_2 \vee \ldots F_m}(r)$

- vjerojatnost da n-torka ne zadovoljava pojedinačni uvjet F_i je $1 - N_{F_i}(r) / N(r)$
  - odnosno $1 - f(F_i, r)$
- ako su F_i nezavisni, vjerojatnost da n-torka ne zadovoljava niti jedan F_i jest

  $$(1 - N_{F_1}(r) / N(r)) \cdot (1 - N_{F_2}(r) / N(r)) \ldots \cdot (1 - N_{F_m}(r) / N(r))$$
  - odnosno $(1 - f(F_1, r)) \cdot (1 - f(F_2, r)) \cdot \ldots \cdot (1 - f(F_m, r))$
- vjerojatnost da zadovoljava barem jedan od F_i

  $$1 - (1 - N_{F_1}(r) / N(r)) \cdot (1 - N_{F_2}(r) / N(r)) \ldots \cdot (1 - N_{F_m}(r) / N(r))$$
  - odnosno $1 - (1 - f(F_1, r)) \cdot (1 - f(F_2, r)) \cdot \ldots \cdot (1 - f(F_m, r))$
- procijenjeni broj n-torki u rezultatu je dakle

  $$N(r) \cdot (1 - (1 - N_{F_1}(r) / N(r)) \cdot (1 - N_{F_2}(r) / N(r)) \ldots \cdot (1 - N_{F_m}(r) / N(r)))$$
  - odnosno $N(r) (1 - (1 - f(F_1, r)) \cdot (1 - f(F_2, r)) \cdot \ldots \cdot (1 - f(F_m, r)))$

Spajanje

$r(R) \triangleright\triangleleft s(S)$

- ako je $R \cap S = \varnothing$ $\Rightarrow N(r) \cdot N(s)$
- ako je $R \cap S =$ ključ u R $\Rightarrow N(s)$ (jer se svaka t_s može spojiti s najviše jednom t_r)
- ako je $R \cap S =$ ključ u S $\Rightarrow N(r)$ (jer se svaka t_r može spojiti s najviše jednom t_s)
- ako je $R \cap S = A$, A je atribut ili skup atributa koji nije ključ niti za R niti za S
  - procjena za $r \triangleright\triangleleft s$
  - promatra se jedna n-torka t_r (ona ima neku konkretnu A-vrijednost)
  - broj n-torki t_s koje imaju A-vrijednost jednaku A-vrijednosti dotične n-torke t_r
    $= N(s) / V(A, s)$
  - svaka t_r će se spojiti sa $N(s) / V(A, s)$ n-torki t_s. Procijenjeni broj n-torki
    $N(r \triangleright\triangleleft s) = N(r) \cdot N(s) / V(A, s)$
  - simetrično: $N(s \triangleright\triangleleft r) = N(s) \cdot N(r) / V(A, r)$
  - očito, ako je $V(A, r) \neq V(A, s)$, procjene se razlikuju! U takvom slučaju, kao konačna procjena uzima se manja od dviju prikazanih, dakle
    $N(r \triangleright\triangleleft s) = N(r) \cdot N(s) / \max(V(A, r), V(A, s))$

Unija, presjek, razlika, projekcija, agregacija

$\sigma_{F_1}(r) \cup \sigma_{F_2}(r)$ unija podskupova iste relacije

- procjenjuje se jednako kao rezultat operacije $\sigma_{F_1 \vee F_2}(r)$

$\sigma_{F_1}(r) \cap \sigma_{F_2}(r)$ presjek podskupova iste relacije

- procjenjuje se jednako kao rezultat operacije $\sigma_{F_1 \wedge F_2}(r)$

$r \cup s$	$\Rightarrow$	$N(r) + N(s)$
$r \cap s$	$\Rightarrow$	$\min(N(r), N(s))$
$r \setminus s$	$\Rightarrow$	$N(r)$
$\pi_L(r)$	$\Rightarrow$	$V(L, r)$
${}_L\mathcal{G}_{\mathcal{AF}_1(B_1), \mathcal{AF}_2(B_2), ..., \mathcal{AF}_n(B_n)}(r)$	$\Rightarrow$	$V(L, r)$

Primjer

```
CREATE TABLE ispit (
  mbrStud
, sifPred
, sifNast
, datIspit
, ocjena);
```

N(ispit) = 200 000
V(ocjena, ispit) = 5
V(datIspit, ispit) = 1500
V(sifNast, ispit) = 100
V(sifPred, ispit) = 200
min(sifNast, ispit) = 101
max(sifNast, ispit) = 200

- procijeniti broj n-torki u rezultatu operacije:

$\sigma_{\text{ocjena=2} \wedge \text{datIspit='16.4.2008'}}(\text{ispit})$

$N(\sigma_{\text{ocjena=2}}(\text{ispit})) = N(\text{ispit}) / V(\text{ocjena, ispit})$

$N(\sigma_{\text{datIspit='16.4.2008'}}(\text{ispit})) = N(\text{ispit}) / V(\text{datIspit, ispit})$

$$N(r) \cdot (N_{F_1}(r) \cdot N_{F_2}(r) \cdot \ldots \cdot N_{F_m}(r)) \; / \; (N(r))^m$$

$N(\text{rez}) = N(\text{ispit}) / (V(\text{ocjena, ispit}) \cdot V(\text{datIspit, ispit})) \approx 27$

- za vježbu (ovaj i sljedeća dva zadatka) riješite korištenjem notacije selektivnosti predikata

Primjer

```
CREATE TABLE ispit (
  mbrStud
, sifPred
, sifNast
, datIspit
, ocjena);
```

N(ispit) = 200 000
V(ocjena, ispit) = 5
V(datIspit, ispit) = 1500
V(sifNast, ispit) = 100
V(sifPred, ispit) = 200
min(sifNast, ispit) = 101
max(sifNast, ispit) = 200

- procijeniti broj n-torki u rezultatu operacije:

$\sigma_{ocjena=2 \wedge sifNast \leq 120}(ispit)$

$N(\sigma_{ocjena=2}(ispit)) = N(ispit) / V(ocjena, ispit) = 40\,000$

$N(r) \cdot (v - min(A, r)) / (max(A, r) - min(A, r))$

$N(\sigma_{sifNast \leq 120}(ispit)) = N(ispit) \cdot 19 / 99 \approx 38\,380$

$N(r) \cdot (N_{F_1}(r) \cdot N_{F_2}(r) \cdot \ldots \cdot N_{F_m}(r)) / (N(r))^m$

$N(rez) \approx N(ispit) \cdot (40\,000 \cdot 38\,380) / N(ispit)^2 \approx 7677$

Primjer

```
CREATE TABLE ispit (
  mbrStud
, sifPred
, sifNast
, datIspit
, ocjena);
```

N(ispit) = 200 000
V(ocjena, ispit) = 5
V(datIspit, ispit) = 1500
V(sifNast, ispit) = 100
V(sifPred, ispit) = 200
min(sifNast, ispit) = 101
max(sifNast, ispit) = 200

- procijeniti broj n-torki u rezultatu operacije:

$\sigma_{sifNast=120 \vee sifPred=320}(ispit)$

$N(\sigma_{sifNast=120}(ispit)) = N(ispit) / V(sifNast, ispit) = 2\,000$

$N(\sigma_{sifPred=320}(ispit)) = N(ispit) / V(sifPred, ispit) = 1\,000$

$$N(r) \cdot (1 - (1 - N_{F_1}(r) / N(r)) \cdot (1 - N_{F_2}(r) / N(r)) \ldots \cdot (1 - N_{F_m}(r) / N(r)))$$

$N(rez) = N(ispit) \cdot (1 - (1 - 2000 / 200000) \cdot (1 - 1000 / 200000)) = 2990$

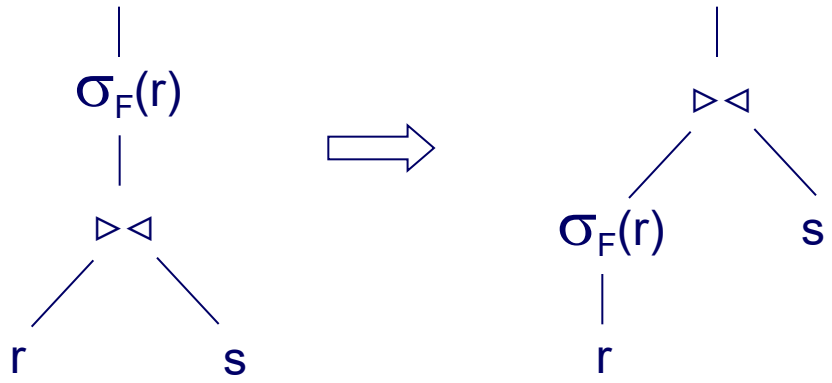
Optimizacija

Tehnike optimizacije

- **optimizacija temeljena na primjeni heurističkih pravila**
  - *heuristic optimization*
  - iz relacijske algebre poznata su pravila ekvivalencije (*equivalence rules*) kojima se jedan izraz relacijske algebre može transformirati u ekvivalentni izraz relacijske algebre
  - ovdje će se koristiti ona pravila ekvivalencije čijom se primjenom, u većini slučajeva (ali ne uvijek!), dobije bolji plan izvršavanja

Primjer: oznake: r(R) relacija r sa shemom R; s(S) relacija s sa shemom S

- potiskivanje selekcije: operaciju selekcije obaviti u čim ranijoj fazi
- ako F sadrži samo atribute iz R, tada $\sigma_F (r \bowtie s) \equiv \sigma_F (r) \bowtie s$



Tehnike optimizacije

- **optimizacija temeljena na procjeni troškova**
  - *cost based optimization*
  - optimizator izračunava ukupni trošak svakog ekvivalentnog plana izvršavanja (ili mnogih ekvivalentnih planova). Pri tome procjenjuje:
    - broj U/I operacija
    - veličina korištene glavne i sekundarne memorije
    - trošak procesorskog vremena
    - komunikacijski troškovi (za distribuirane baze podataka)
  - odabire plan izvršavanja s najmanjim procijenjenim ukupnim troškom
  - nedostatak: procjena troška obavlja se za veliki broj alternativnih planova izvršavanja. Trošak same optimizacije postaje značajan faktor

- kao jednostavan primjer optimizacije temeljene na procjeni troškova može poslužiti primjer procjene troškova u složenim operacijama relacijske algebre. U primjeru su troškovi procijenjeni za (samo) tri alternativna plana, te se usporedbom ukupnog troška moglo procijeniti koji je plan najbolji

Tehnike optimizacije

- najčešće se se ove dvije tehnike kombiniraju: primjenom heurističkih pravila se odredi ograničeni broj "potencijalno dobrih" planova izvršavanja, te se na tako ograničenom skupu planova provodi optimizacija temeljena na procjeni troškova
- sustavi za upravljanje bazama podataka omogućuju određeni stupanj kontrole nad odabirom tehnike optimizacije

Primjer:

- *The algorithm that a SET OPTIMIZATION HIGH statement invokes is a sophisticated, cost-based strategy that examines all reasonable query plans and selects the best overall alternative. For large joins, this algorithm can incur more overhead than desired. In extreme cases, you can run out of memory.*
- *The alternative algorithm that a SET OPTIMIZATION LOW statement invokes eliminates unlikely join strategies during the early stages, which reduces the time and resources spent during optimization. However, when you specify a low level of optimization, the optimal strategy might not be selected because it was eliminated from consideration during early stages of the algorithm.*

IBM Informix Dynamic Server Performance Guide

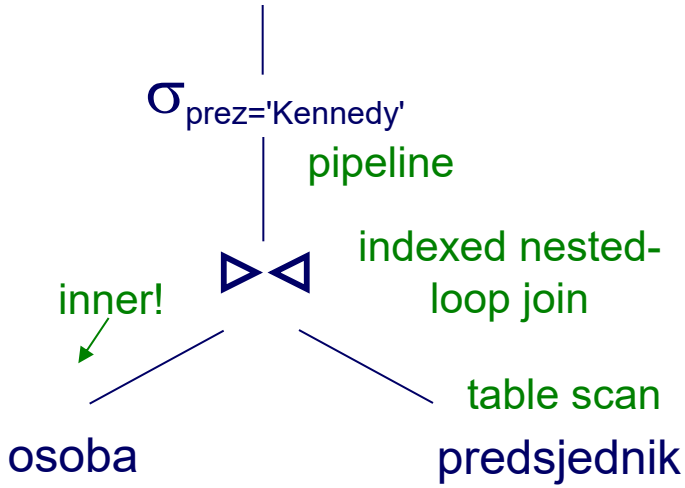
Samo heuristička optimizacija nije dovoljna

- optimizator se ne bi trebao osloniti samo na pravila za heurističku optimizaciju
- primjer pokazuje kako se nekritičkom primjenom pravila heurističke optimizacije može dobiti loš plan

```
SELECT * FROM osoba JOIN predsjednik
  ON osoba.sif = predsjednik.sifOso
  WHERE prez = 'Kennedy';
```

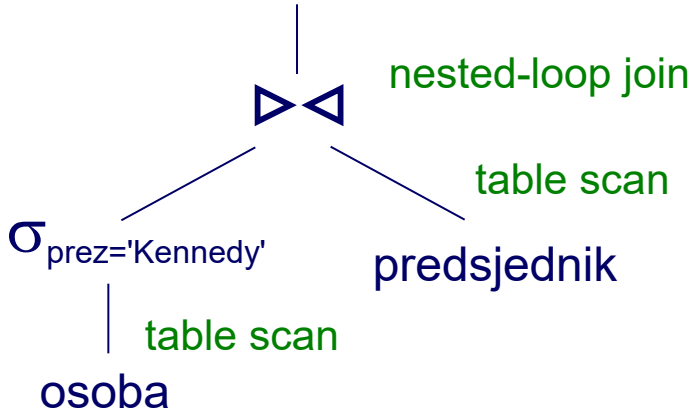
- N(osoba) = 1 000 000
- N(predsjednik) = 20

- V(prez, osoba) = 5 000
- idx za osoba.sif, d(idx) = 4
- B(osoba) = 10 000
- B(predsjednik) = 1



ukupno: ≈ 80 **U/I operacija**

potiskivanje selekcije



ukupno: ≈ 10 000 **U/I operacija**

Heuristička optimizacija

Pravila za transformaciju izraza relacijske algebre

- transformacije se temelje na pravilima ekvivalentnosti izraza relacijske algebre
- *equivalence rules, algebraic laws*
- ovdje će biti navedena samo najvažnija pravila za transformaciju, ona koja su važna u jednostavnijim postupcima heurističke optimizacije

- oznake
  - relacije r(R), s(S), t(T), ...
  - formule F, F_1, F_2, F_3, G, ...

1. Pravila vezana uz operaciju selekcije

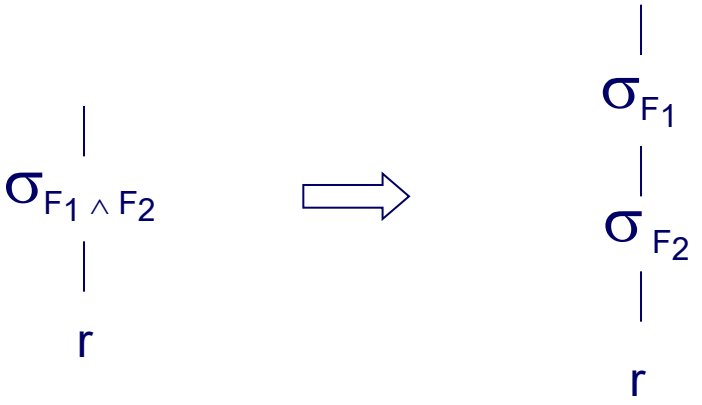
1.1. $\sigma_{F_1 \wedge F_2}(r) \equiv \sigma_{F_1}(\sigma_{F_2}(r))$ — dekompozicija operatora selekcije s konjunktivnim oblikom predikata

1.2. $\sigma_{F_1}(\sigma_{F_2}(r)) \equiv \sigma_{F_2}(\sigma_{F_1}(r))$ — promjena redoslijeda operacija selekcije

Primjena na stablu upita

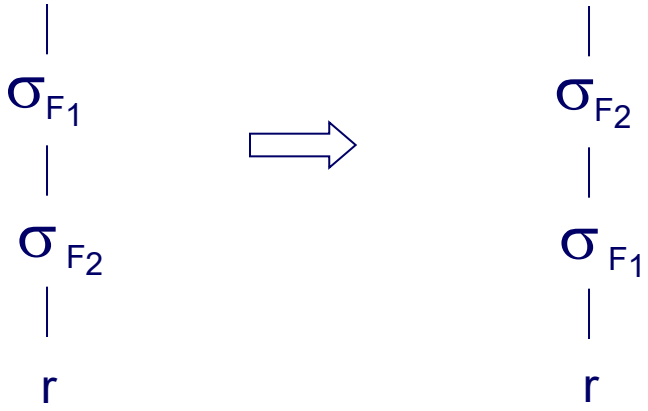
dekompozicija operatora selekcije s konjunktivnim oblikom formule

$$\sigma_{F_1 \wedge F_2}(r) \equiv \sigma_{F_1}(\sigma_{F_2}(r))$$



promjena redoslijeda operacija selekcije

$$\sigma_{F_1}(\sigma_{F_2}(r)) \equiv \sigma_{F_2}(\sigma_{F_1}(r))$$



Pravila za transformaciju izraza relacijske algebre

- r(R), s(S). Ako F sadrži samo atribute iz R

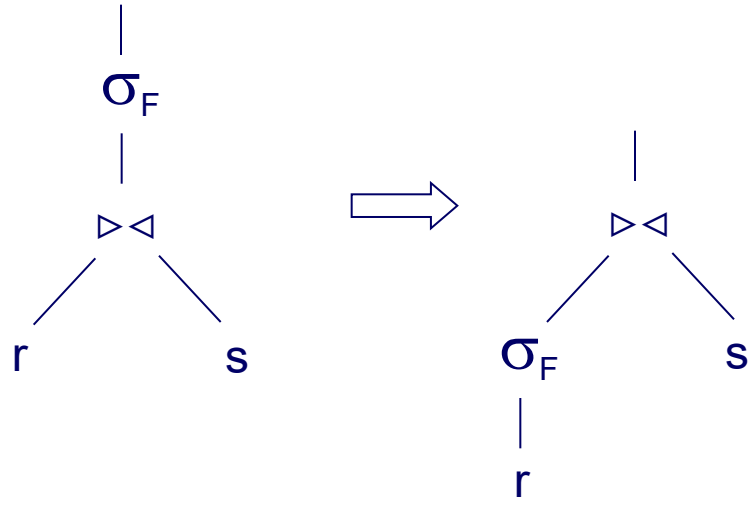
1.3. $\sigma_F (r \bowtie s) \equiv \sigma_F (r) \bowtie s$

1.4. $\sigma_F (r \underset{G}{\bowtie} s) \equiv \sigma_F (r) \underset{G}{\bowtie} s$

1.5. $\sigma_F (r \times s) \equiv \sigma_F (r) \times s$

Prikaz često korištene transformacije: potiskivanje selekcije

$\sigma_F (r \bowtie s) \equiv \sigma_F (r) \bowtie s$

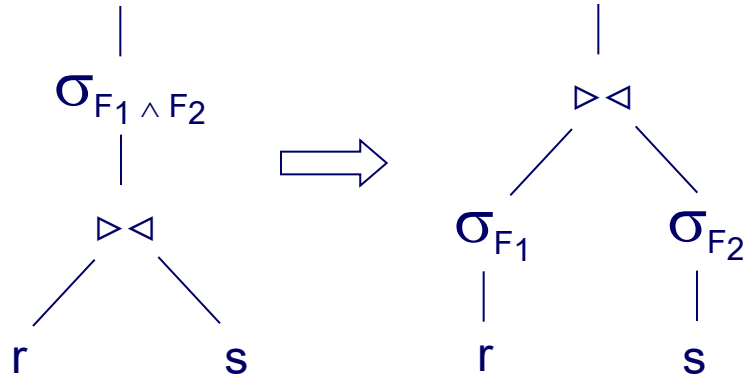


Pravila za transformaciju izraza relacijske algebre

- slično, ako F_1 sadrži samo atribute iz R, a F_2 sadrži samo atribute iz S

1.6. $\sigma_{F_1 \wedge F_2}(r \bowtie s) \equiv \sigma_{F_1}(r) \bowtie \sigma_{F_2}(s)$

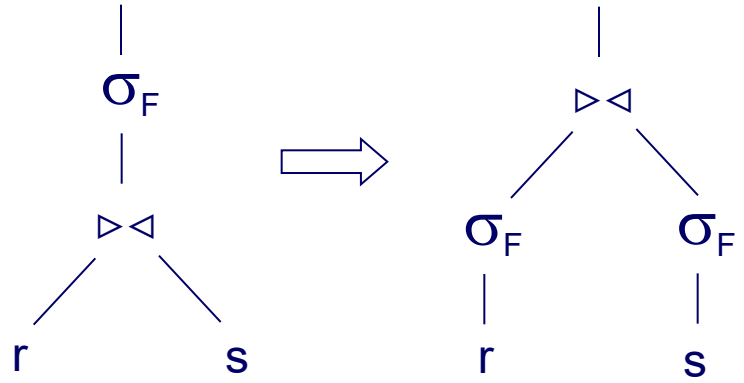
1.7. $\sigma_{F_1 \wedge F_2}(r \underset{G}{\bowtie} s) \equiv \sigma_{F_1}(r) \underset{G}{\bowtie} \sigma_{F_2}(s)$



- slično, <u>ako</u> **i** R **i** S sadrže sve atribute iz F

1.8. $\sigma_F(r \bowtie s) \equiv \sigma_F(r) \bowtie \sigma_F(s)$

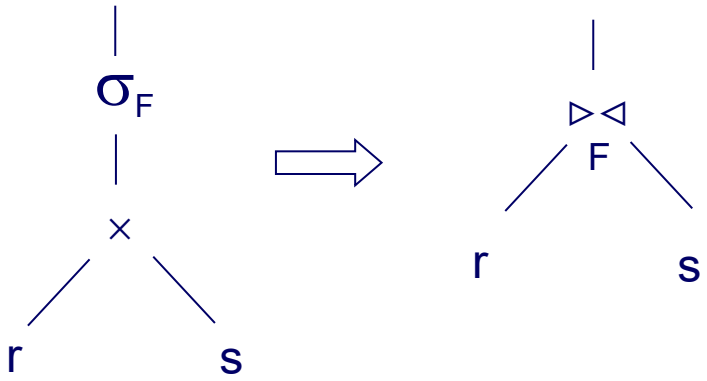
1.9. $\sigma_F(r \underset{G}{\bowtie} s) \equiv \sigma_F(r) \underset{G}{\bowtie} \sigma_F(s)$



Pravila za transformaciju izraza relacijske algebre

- važno pravilo o selekciji, Kartezijevom produktu i spajanju

1.10. $\sigma_F (r \times s) \equiv r \underset{F}{\triangleright\triangleleft} s$



Pravila za transformaciju izraza relacijske algebre

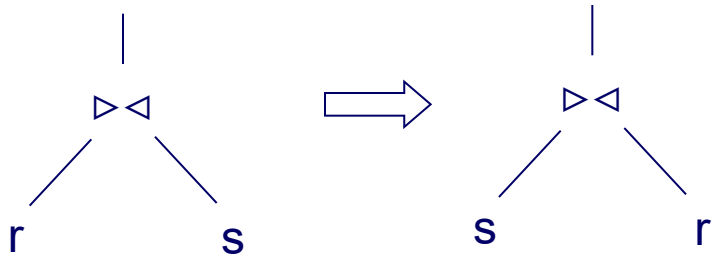
2. Komutativnost

2.1. $r \times s \equiv s \times r$

2.2. $r \rhd\lhd s \equiv s \rhd\lhd r$

2.3. $r \rhd\lhd_F s \equiv s \rhd\lhd_F r$

Primjer: slobodna promjena redoslijeda operanada



Pravila za transformaciju izraza relacijske algebre

3. Asocijativnost

3.1. $(r \times s) \times t \equiv r \times (s \times t)$

3.2. $(r \rhd\lhd s) \rhd\lhd t \equiv r \rhd\lhd (s \rhd\lhd t)$

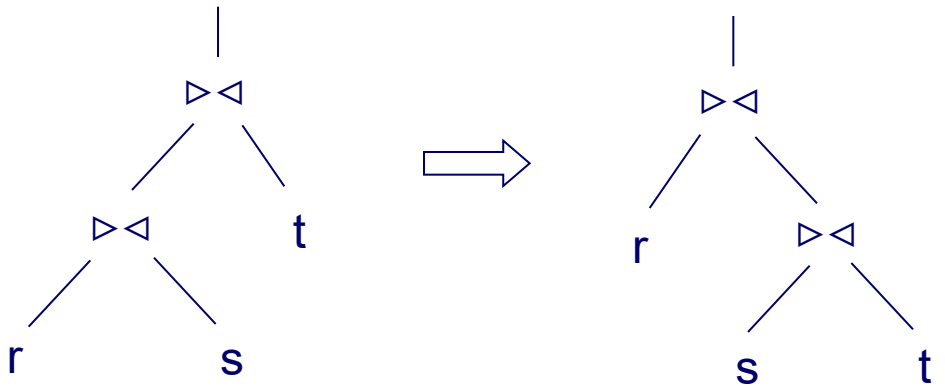
Važno

3.3. $(r \underset{F_1}{\rhd\lhd} s) \underset{F_2 \wedge F_3}{\rhd\lhd} t \equiv r \underset{F_1 \wedge F_3}{\rhd\lhd} (s \underset{F_2}{\rhd\lhd} t)$ uz uvjet da F_2 sadrži samo atribute iz S i T

Informativno

Primjer: često korištena transformacija: promjena redoslijeda spajanja

$(r \rhd\lhd s) \rhd\lhd t \equiv r \rhd\lhd (s \rhd\lhd t)$



Osnovni koraci u heurističkoj optimizaciji upita

1. dekomponirati operatore selekcije koji sadrže konjunktivne izraze
   - postiže se veći stupanj slobode pomicanja operatora selekcije unutar stabla upita
2. potisnuti operatore selekcije što je moguće dublje prema listovima stabla upita
   - obavljanjem operacija selekcije u najranijim mogućim fazama smanjuje se veličina međurezultata koji se koristi u daljnjim operacijama što najčešće dovodi do smanjenja ukupnog troška
3. Kartezijev produkt kombinirati s operacijom selekcije u operaciju spajanja uz uvjet (θ-join)
4. redoslijed spajanja odrediti tako da se prvo obavljaju operacije spajanja koje rezultiraju najmanjim brojem n-torki (ili veličinom međurezultata)

- ovdje su navedena tek najvažnija (najčešće korištena) pravila

Navedena pravila koristiti za rješavanje zadataka u kojima se traži heuristička optimizacija

Primjer

```
SELECT *
  FROM zup, mjesto, osoba
  WHERE zup.sifZup = mjesto.sifZup
    AND mjesto.pbr = osoba.pbr
    AND mjesto.pbr = 21000
    AND osoba.prez = 'Novak';
```

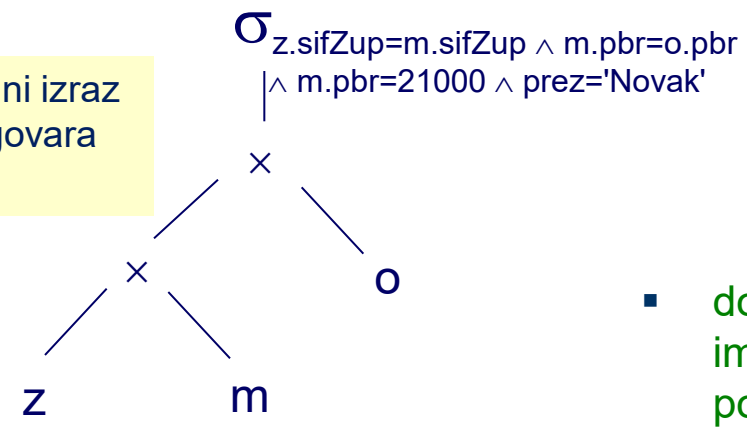
zup
sifZup
nazZup

mjesto
pbr
nazMj
sifZup

osoba
sifOso
nazMj
ime
prez
pbr

- nacrtati inicijalno stablo upita. Redoslijed operacija Kartezijevog produkta (uočiti da originalni SQL upit ne sadrži operacije spajanja!) u inicijalnom stablu mora odgovarati redoslijedu kojim su relacije navedene u upitu
- provesti heurističku optimizaciju

- za vježbu napišite i inicijalni izraz relacijske algebre koji odgovara inicijalnom stablu upita

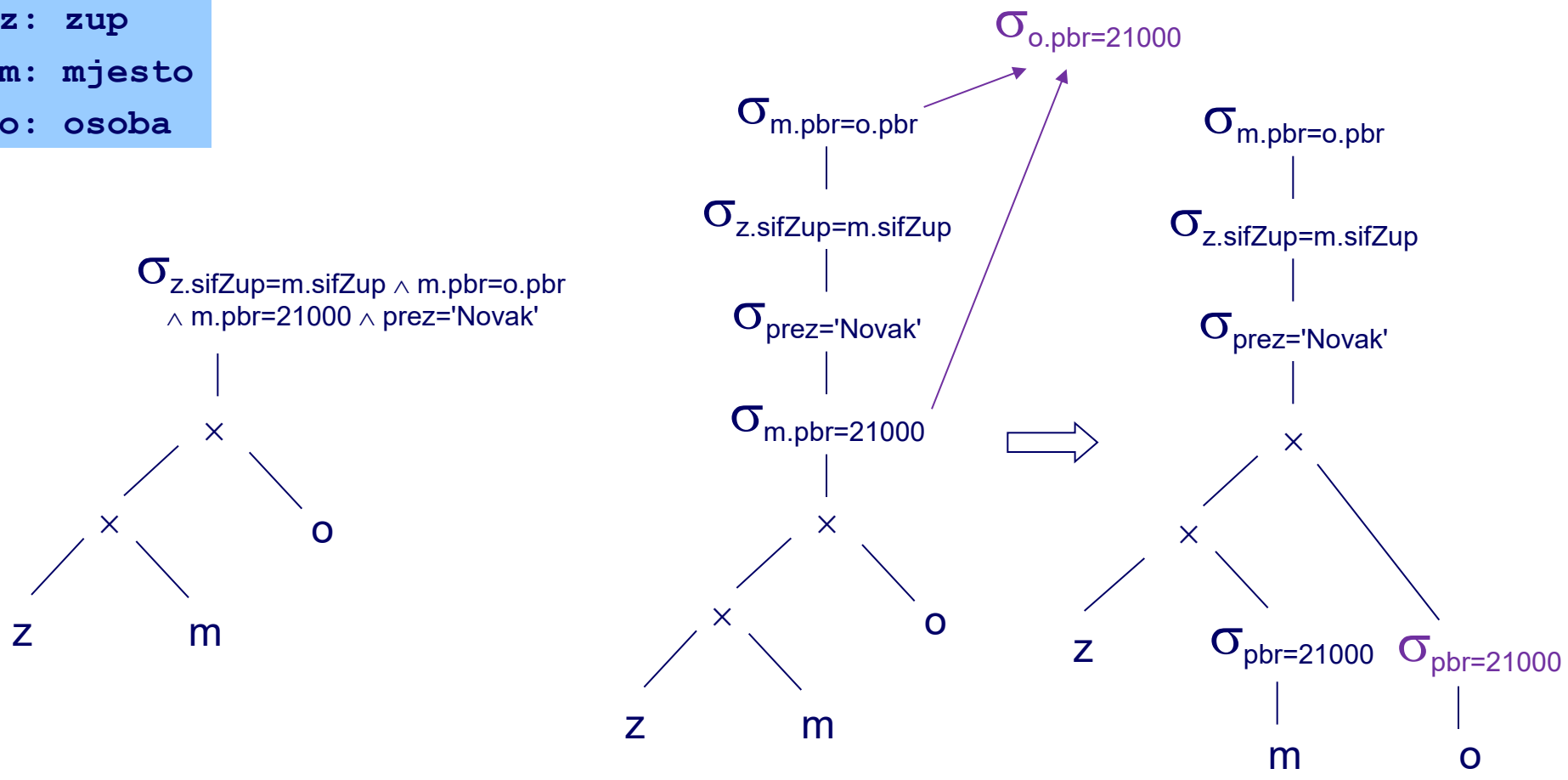



```
z: zup
m: mjesto
o: osoba
```

- dogovor: tamo gdje je to nužno, imena atributa se kvalificiraju pomoću *imeTablice.imeAtributa*

Primjer

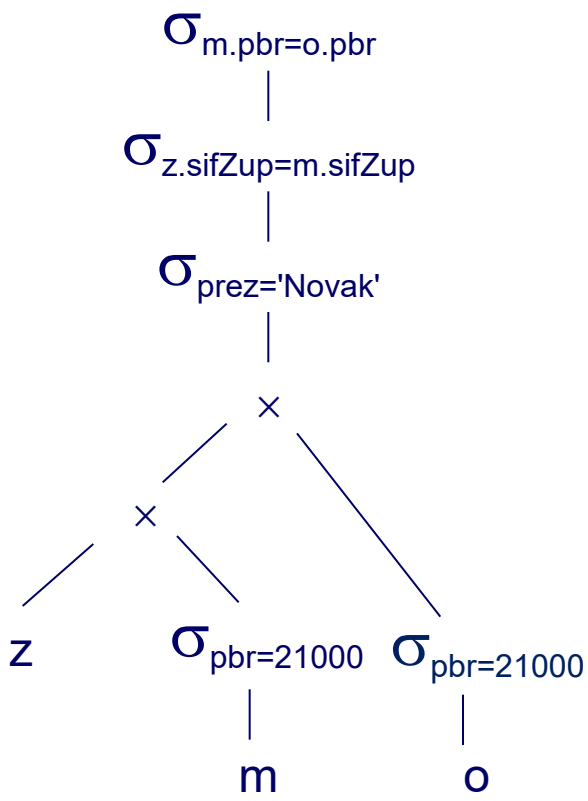
```
z: zup
m: mjesto
o: osoba
```

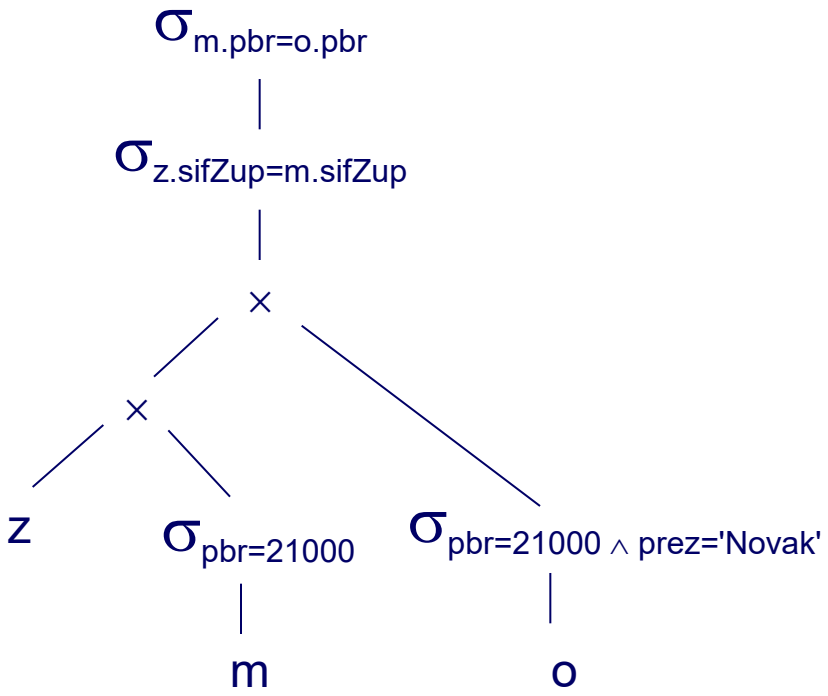


$m.pbr=o.pbr \wedge m.pbr=21000 \Rightarrow o.pbr=21000$

Primjer

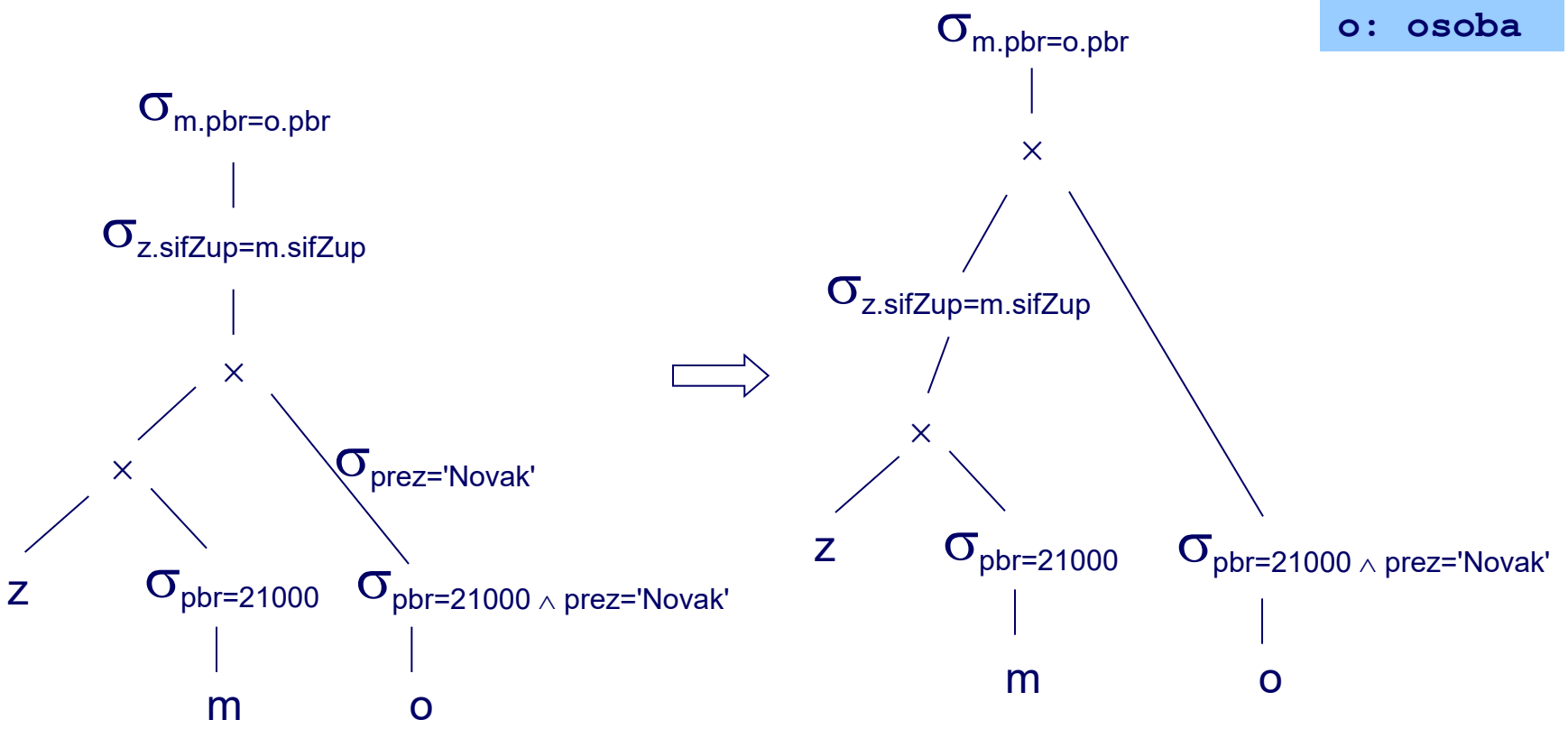
```
z: zup
m: mjesto
o: osoba
```





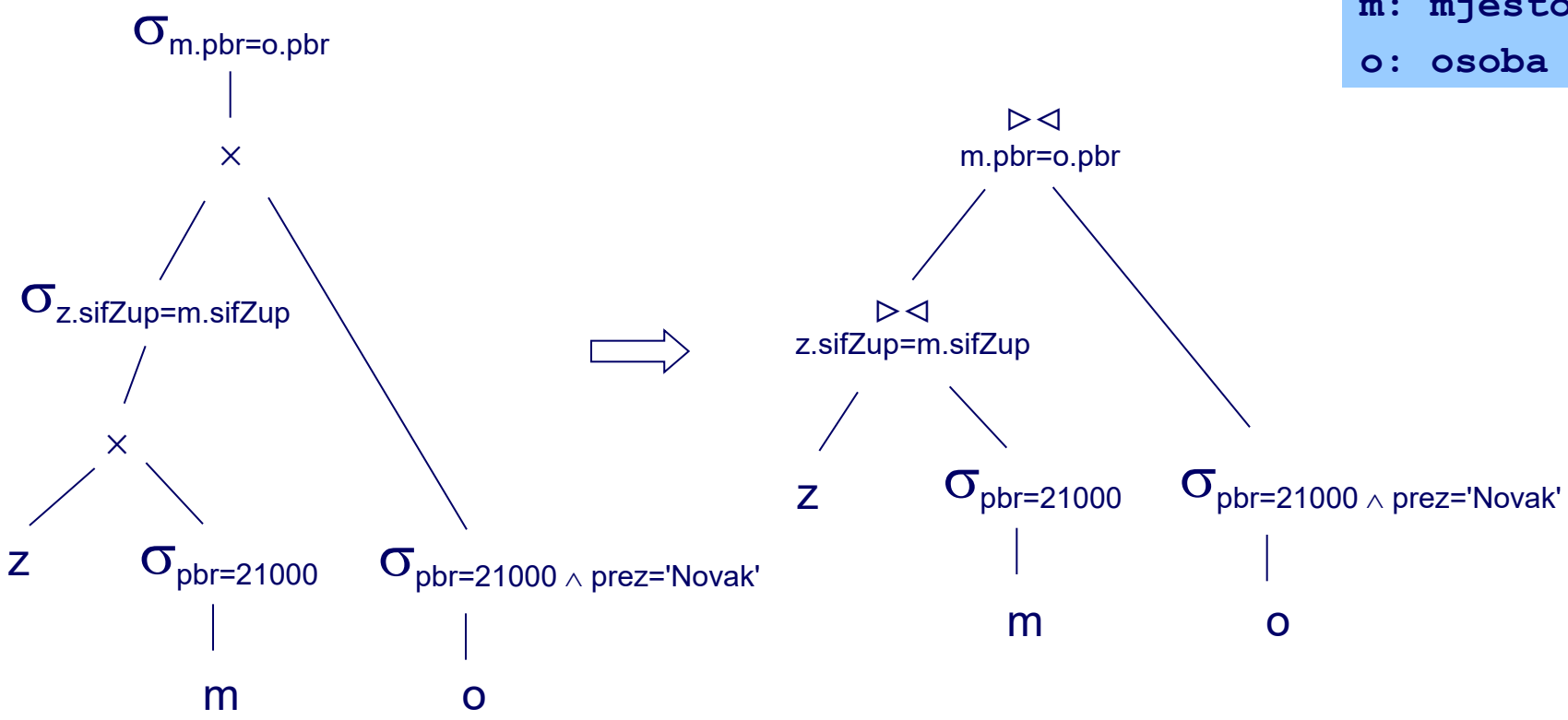
Primjer

```
z: zup
m: mjesto
o: osoba
```



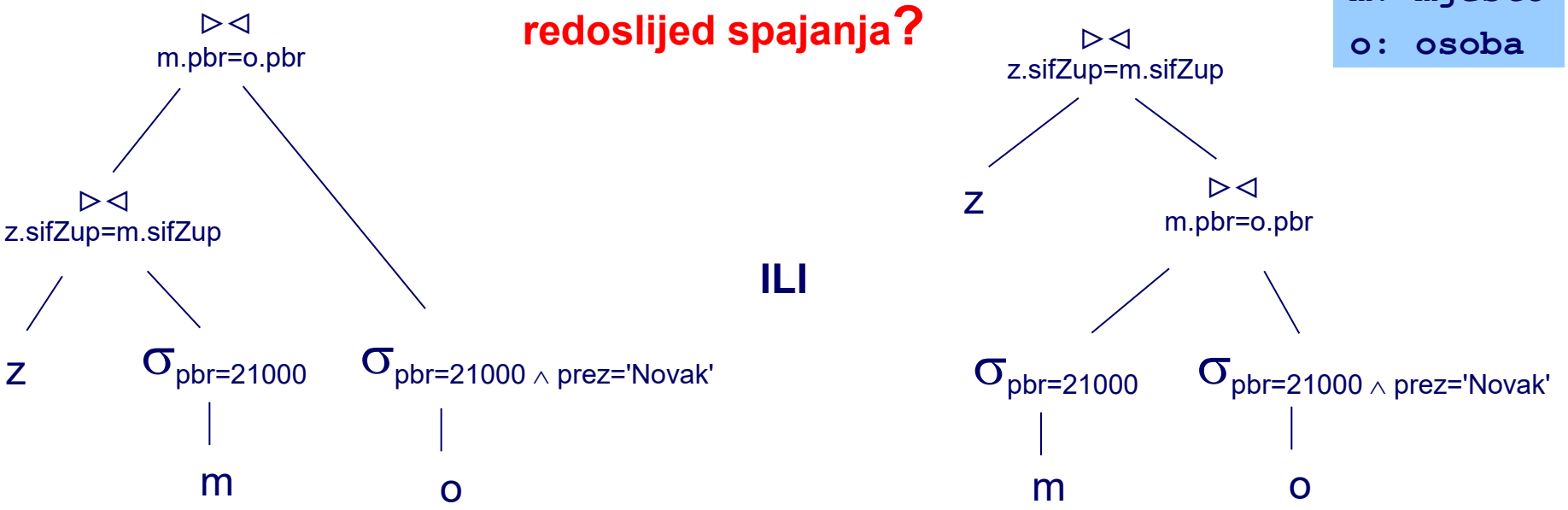
Primjer

```
z: zup
m: mjesto
o: osoba
```



Primjer

```
z: zup
m: mjesto
o: osoba
```



- U četvrtom koraku, za koji je međutim već potrebno uzeti u obzir troškove izvršavanja operacija, optimizator bi trebao izračunati trošak obavljanja ekvivalentnih planova dobivenih primjenom pravila o asocijativnosti spajanja. S obzirom da je za to potreban opsežan skup statističkih podataka, to se ovdje neće provoditi.

Primjer prikaza plana izvršavanja (IBM Informix)

Informativno

```
SELECT *
   FROM ispit
      , stud
      , mjesto
   WHERE mjesto.pbr = stud.pbrStan
     AND stud.mbr = ispit.mbrStud
     AND mjesto.pbr = 10805
     AND ispit.ocjena = 1;

Estimated Cost: 8409
Estimated # of Rows Returned: 42

  1) informix.stud: SEQUENTIAL SCAN
        Filters: informix.stud.pbrstan = 10805
  2) informix.mjesto: SEQUENTIAL SCAN
        Filters:
        Table Scan Filters: informix.mjesto.pbr = 10805
DYNAMIC HASH JOIN
    Dynamic Hash Filters: informix.mjesto.pbr = informix.stud.pbrstan
  3) informix.ispit: SEQUENTIAL SCAN
        Filters:
        Table Scan Filters: informix.ispit.ocjena = 1

DYNAMIC HASH JOIN (Build Outer)
    Dynamic Hash Filters: informix.stud.mbr = informix.ispit.mbrstud
```

Primjer prikaza plana izvršavanja (IBM Informix)

Informativno

- Procijenjeni trošak: **8409**
- Procijenjeni broj n-torki u rezultatu: **42**
- (1) *table scan* za stud, uz filter pbrstan = 10805 → međurezultat 1
- (2) *table scan* za mjesto, uz filter pbr = 10805 → međurezultat 2
  - prethodna dva međurezultata (1 i 2) spoji pomoću *hash join* → međurezultat 3
- (3) *table scan* za ispit, uz filter ocjena = 1 → međurezultat 4
  - prethodna dva međurezultata (3 i 4) spoji pomoću *hash join*

Primjer prikaza plana izvršavanja (IBM Informix)

Informativno

```
Query statistics:
-----------------

  Table map :
  ----------------------------
  Internal name     Table name
  ----------------------------
  t1                stud
  t2                mjesto
  t3                ispit

  type     table  rows_prod  est_rows  rows_scan  time       est_cost
  ---------------------------------------------------------------------
  scan     t1     8          16        5000       00:00.01   369

  type     table  rows_prod  est_rows  rows_scan  time       est_cost
  ---------------------------------------------------------------------
  scan     t2     1          1         1000       00:00.00   46

  type     rows_prod  est_rows  rows_bld  rows_prb  novrflo  time       est_cost
  ---------------------------------------------------------------------------
  hjoin    8          9         1         8         0        00:00.01   417

  ...
```

Primjer prikaza plana izvršavanja (IBM Informix)

Informativno

- nakon izvršavanja upita, optimizator u izvještaj o planu izvršavanja dodaje i statističke podatke o upitu, npr.
- za dio koji se odnosi na operaciju *table scan* nad relacijom stud
  - procijenjeni trošak: 369
  - procijenjeni broj n-torki u rezultatu: 16
  - stvarni broj n-torki u rezultatu (poznato tek nakon izvršavanja upita): 8
  - broj pročitanih n-torki: 5000

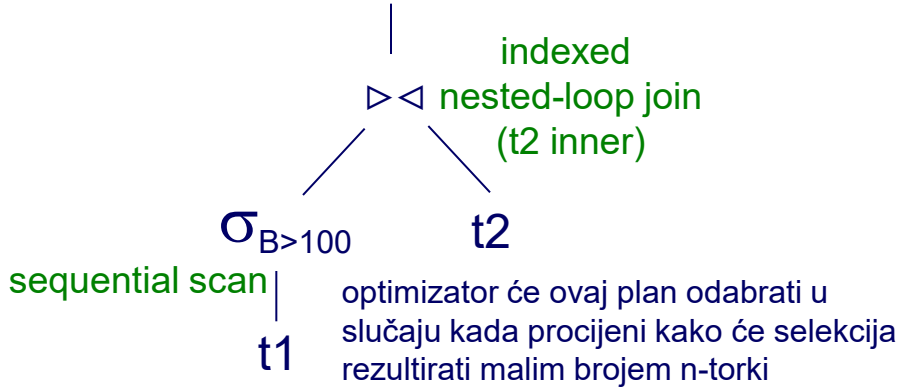
- velika razlika između procijenjenog i stvarnog troška obavljanja operacije, upućuje na to da optimizator na raspolaganju nema dovoljno kvalitetne statističke podatke
  - administrator oko toga treba nešto poduzeti. Npr. kreirati histograme za neke atribute

Upute optimizatoru (*optimizer hints, optimizer directives*)

- mogućnost koju treba koristiti vrlo štedljivo jer:
  1. uvođenjem uputa optimizatoru odgovornost za "optimalno" izvršavanje upita prebacuje se sa SUBP-a na administratora (ili programera ili korisnika)
  2. ne postoji standardna sintaksa

Primjer:

```
SELECT *
   FROM t1 JOIN t2
     ON t1.A = t2.A
   WHERE t1.B > 100;
```



- postoji indeks nad atributom `t2.A`
- ako optimizator procijeni da će uvjet `t1.B > 100` vratiti veliki broj n-torki iz t1, neće koristiti *indexed nested-loop join* i radije primijeniti *hash join*. Zašto?
- ako administrator ili programer (ili vrlo napredni korisnik), prema značenju podataka ocijeni da će rezultat selekcije ipak sadržavati relativno malen broj n-torki, može uz SELECT naredbu optimizatoru uputiti sljedeću direktivu:

```
SELECT {+USE_NL(t2)} *
   FROM t1 JOIN t2
     ON t1.A = t2.A
   WHERE t1.B > 100;
```

- uputa: spajanje s t2 obavi pomoću *nested-loop join* (NL=nested loop)

Literatura:

- H. Garcia-Molina, J. Ullman, J. Widom. Database system implementation. Prentice-Hall International, Inc. Upper Saddle River, New Jersey. 2000.
- A. Silberschatz, H. Korth, S. Sudarshan. Database system concepts, 6th ed. McGraw-Hill. 2010.
- R. Elmasri, S. Navathe. Fundamentals of database systems, 3rd ed. Addison-Wesley Publishing Company. Reading, Massachusetts. 2000.
- IBM Informix Dynamic Server Performance Guide, Version 11.50, IBM, 2008.